



I E S A L B A R R E G A S

TÉCNICO SUPERIOR EN DESARROLLO DE
APLICACIONES WEB

Departamento de Informática

PROYECTO

Eldera

Manual Técnico

Índice

Contenido

Eldera.....	1
Manual Técnico.....	1
1.Introducción	3
2.Arquitectura de la aplicación	5
2.1 Frontend.....	7
2.1.1 Tecnologías usadas.....	7
2.1.2 Entorno de desarrollo	7
2.2 Backend.....	8
2.2.1 Tecnologías usadas.....	8
2.2.2 Entorno de desarrollo	9
3.Documentación técnica.....	9
3.1.1. Análisis.....	9
3.1.2. Diagramas.....	9
3.2. Desarrollo.....	16
3.3. Pruebas realizadas	16
4.Proceso de despliegue	17
5.Propuestas de mejoras.....	18
6.Bibliografía	19

1. Introducción

Eldera es una plataforma avanzada de gestión clínica y administrativa diseñada específicamente para entornos residenciales geriátricos. El objetivo de este sistema es ofrecer un núcleo de software de alta disponibilidad centrado en la seguridad del paciente y la eficiencia operativa del personal multidisciplinar.

Para el diseño de Eldera se han definido los siguientes requisitos clave que garantizan la operatividad técnica y clínica:

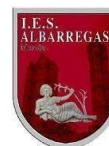
Requisitos funcionales:

- **RF1. Gestión de Sesiones:** El sistema permitirá a los usuarios identificarse mediante usuario y contraseña, recibiendo un token de acceso seguro (JWT).
- **RF2. Gestión de Residentes:** El usuario con permisos podrá Crear, Leer, Actualizar y Eliminar (CRUD) fichas de residentes.
- **RF3. Búsqueda:** El sistema permitirá filtrar residentes por nombre, apellido y número de habitación en tiempo real.
- **RF5. Exportación de Datos:** El sistema permitirá descargar el listado actual de residentes en formato PDF.
- **RF6. Control de Acceso:** El sistema restringirá ciertas acciones (como borrar residentes) únicamente a usuarios con permiso.
- **RF7. Listados Automáticos:** A partir de la ficha de cada residente se podrán crear listados dinámicos que estén actualizados continuamente.

Requisitos No Funcionales:

- **RNF1. Seguridad:** Las contraseñas de los usuarios se almacenarán encriptadas utilizando algoritmos de hashing robustos (Bcrypt).
- **RNF2. Usabilidad/Responsive:** La interfaz se adaptará fluidamente a pantallas de escritorio, tablets y móviles.
- **RNF3. Portabilidad:** La aplicación deberá poder desplegarse en cualquier sistema operativo que soporte Docker sin necesidad de instalar dependencias locales.
- **RNF4. Rendimiento:** Las respuestas de la API no deberán exceder los 200ms para operaciones de lectura estándar.

Proyecto “Desarrollo de Aplicaciones Web”



JUNTA DE EXTREMADURA

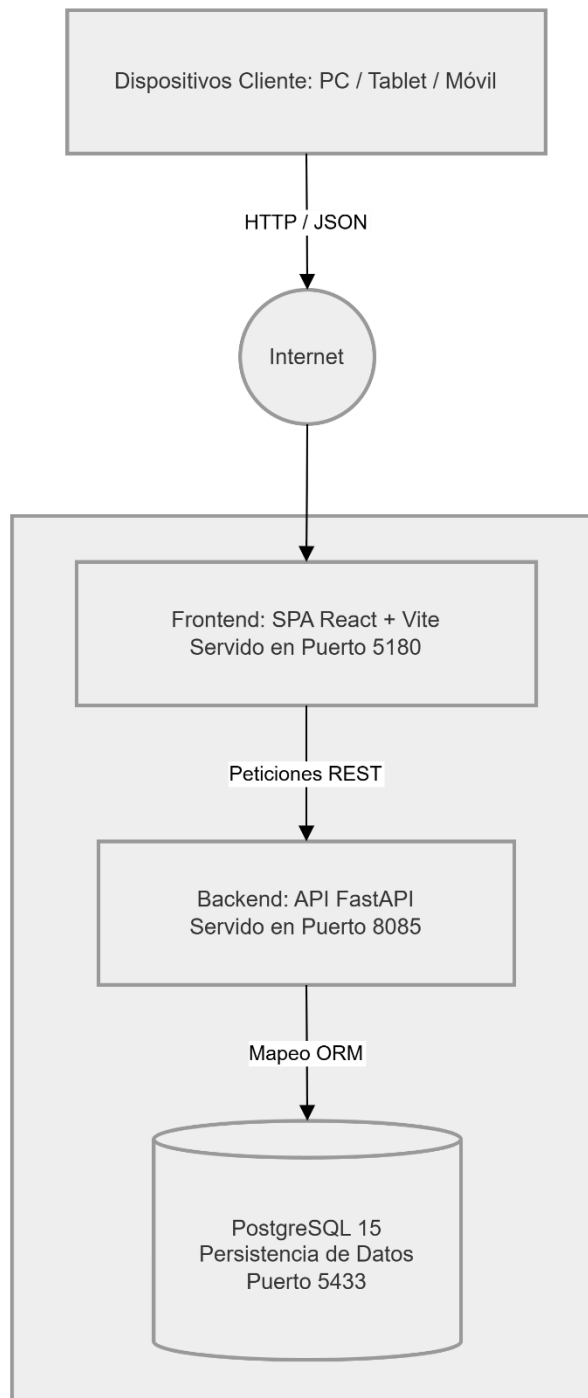
Consejería de Educación y Empleo

Matriz de Responsabilidades (Roles vs Funcionalidades):

Funcionalidad	Admin	Médico	Enfermería	Auxiliar	Fisio	T.O.	T. Social	Psicólogo
Ficha Identificación (Tab 0) y eliminación de residente	✓	-	-	-	-	-	-	-
Alta de Residentes	✓	✓	✓	-	-	-	-	-
Ver Perfil Clínico (Lectura)	✓	✓	✓	✓	✓	✓	✓	✓
Registrar Constantes (Vitals)	✓	✓	✓	-	-	-	-	-
Registrar Cuidados Diarios	✓	✓	✓	✓	-	-	-	-
Modificar Patrones Clínicos	✓	✓	✓	-	-	-	-	-
Movilidad y Marcha (P4)	✓	✓	✓	✓	✓	-	-	-
Cognitivo y MMSE (P6)	✓	✓	✓	-	✓	✓	-	✓
Adaptación y Sujeciones (P10)	✓	✓	✓	-	✓	-	-	-
Valoración Social (P7, P8, P11)	✓	✓	✓	-	-	-	✓	✓
Exportar Listado PDF	✓	✓	✓	✓	✓	✓	✓	✓

2. Arquitectura de la aplicación

El proyecto sigue una arquitectura Cliente-Servidor fuertemente desacoplada, donde el Frontend (SPA) se comunica con el Backend (API RESTful) exclusivamente mediante peticiones HTTP asíncronas con formato JSON.



Estructura del Proyecto:

```
eldera-daw/
├── src/
│   ├── backend/          # Lógica API (FastAPI)
│   │   ├── alembic/      # Migraciones de base de datos
│   │   ├── routers/      # Endpoints clínicos y administrativos
│   │   ├── models.py     # Esquemas base (SQLAlchemy)
│   │   ├── models_extended.py # Esquemas relacionales complejos
│   │   ├── schemas_extended.py # Validaciones Pydantic
│   │   ├── permissions.py # Lógica de autorización RBAC
│   │   └── main.py       # Punto de entrada y middlewares
│   └── frontend/         # Interfaz de usuario (React)
│       ├── src/components/ # Componentes reutilizables UI
│       ├── src/pages/     # Vistas (Dashboard, Perfil, etc.)
│       └── public/        # Recursos estáticos (Logos e iconos)
├── docker-compose.yml    # Orquestación de contenedores
├── render.yaml           # Configuración de despliegue en Render
├── pyproject.toml        # Configuración de dependencias Python (uv)
├── package-lock.json     # Gestión de dependencias Node.js
└── .gitignore            # Exclusión de archivos para control de versiones
```

2.1 Frontend

2.1.1 Tecnologías usadas

- **React (v18 / v19):** Biblioteca principal para la construcción de interfaces de usuario mediante componentes funcionales y Hooks (`useState`, `useEffect`). Elegido por su inmenso ecosistema, enorme comunidad de soporte y rendimiento óptimo en el DOM virtual, ideal para una Single Page Application (SPA) dinámica y reactiva.
- **Vite (v7.x):** Herramienta de compilación (Bundler) ultrarrápida. Se seleccionó frente a alternativas tradicionales como Webpack por sus tiempos de recarga en caliente (HMR) casi instantáneos, lo que agiliza significativamente la experiencia de desarrollo.
- **Tailwind CSS (v4.1+):** Framework CSS basado en utilidades. Permite un diseño de interfaces adaptativo ("Responsive") extremadamente rápido, manteniendo el código CSS modular y libre de archivos de estilos monolíticos.
- **Axios (v1.x+):** Cliente HTTP basado en promesas. Se utilizó para estandarizar las peticiones a la API RESTful debido a su facilidad para configurar interceptores globales (esenciales para inyectar tokens JWT y manejar errores 401 de autorización) frente a Fetch API.
- **Lucide React:** Librería de iconografía vectorial limpia y moderna. Se seleccionó por su diseño minimalista y su peso ligero en el "bundle" final de la aplicación.

2.1.2 Entorno de desarrollo

- **Node.js (v20 LTS):** Entorno de ejecución para las herramientas de frontend.

Proyecto “Desarrollo de Aplicaciones Web”



- **Visual Studio Code:** Editor de código principal.
- Navegadores modernos (Chrome) usando **React Developer Tools**.

2.2 Backend

2.2.1. Tecnologías usadas

- **FastAPI (v0.110+ / Python 3.11+):** Framework web asíncrono. Se eligió por su altísimo rendimiento (basado en Starlette), soporte nativo para programación concurrente (`async`/await``) y la generación automática de documentación interactiva (Swagger UI), facilitando la integración Frontend-Backend.
- **SQLAlchemy (v2.0+):** ORM (Object-Relational Mapper). Traduce filas de base de datos a objetos de Python de forma segura. Se utilizó para prevenir vulnerabilidades como inyecciones SQL y abstraer consultas complejas.
- **Alembic (v1.13+):** Herramienta de migraciones de datos. Integrada con SQLAlchemy, es indispensable para llevar un control de versiones de los cambios en el esquema de la base de datos a lo largo del tiempo sin pérdida de información.
- **PostgreSQL (v15+):** Sistema de Gestión de Bases de Datos Relacional. Elegido por su robustez, cumplimiento estricto del estándar ACID y soporte para tipos de datos JSON (útiles para atributos dinámicos). Es el estándar de oro en sistemas fiables de código abierto.
- **Pydantic (v2.x):** Librería de validación y parseo de datos. Garantiza mediante validación estática que todas las peticiones a la API cumplen con el formato esperado, rechazando datos corruptos o maliciosos antes de que lleguen a la base de datos.

- **Uvicorn (v0.27+):** Servidor web ASGI. Necesario para servir aplicaciones Python asíncronas, aprovechando al máximo la arquitectura orientada a eventos.

2.2.2 Entorno de desarrollo

- **uv:** Gestor de paquetes y dependencias para Python desarrollado en Rust. Seleccionado por ser significativamente más rápido que `pip`.
- **Docker & Docker Compose:** Utilizados para crear contenedores aislados que garantizan que la aplicación corra de la misma manera en desarrollo que en producción.
- **Ruff:** Linter y formateador de código Python extremadamente rápido para mantener el código limpio bajo el estándar PEP-8.

3. Documentación técnica

3.1.1. Análisis

La aplicación parte del principio de aislar las reglas clínicas de negocio de la infraestructura tecnológica. El modelo Entidad-Relación (lógico) centraliza la tabla `Resident`, la cual tiene relaciones principales (One-to-Many) con módulos satélites.

Además, los datos médicos profundos y los 11 patrones funcionales se estructuran extendiendo la información del residente, y se validan en los endpoints del servidor mediante esquemas complejos de Pydantic.

3.1.2. Diagramas

Para comprender la estructura de datos, hemos definido las siguientes entidades principales:

- **`Residence`:** Representa la entidad de nivel superior en nuestro sistema multi-tenancy, almacenando los datos fiscales y el plan de suscripción de cada centro residencial.

Web”

- **`User`**: Gestiona las credenciales y el control de acceso basado en roles (RBAC) de los profesionales, vinculando a cada usuario con su residencia correspondiente.
- **`Resident`**: Es el núcleo del sistema; almacena la ficha técnica, datos administrativos, antecedentes médicos y el estado actual (activo, hospitalizado, etc.) de cada usuario.
- **`ResidentVitalSign`**: Almacena la historia clínica de constantes vitales (Tensión Arterial, Frecuencia Cardíaca, Saturación de Oxígeno, Glucemia, etc.) con trazabilidad temporal.
- **`ResidentCareLog`**: Registra las acciones de cuidados diarios y necesidades básicas (Higiene, Nutrición, Diuresis, cambios posturales, etc.).
- **`ResidentVaccination`**: Detalla el plan de inmunización completo, incluyendo dosis, números de lote y fechas de vencimiento de las vacunas administradas.
- **`ResidentFollowUp`**: Centraliza las notas de seguimiento y evolución clínica cronológica escritas por los diferentes perfiles profesionales.

Destaca la relación de uno a muchos (1 a N) entre 'Resident' y sus entidades clínicas. Esta rigurosa normalización garantiza que un residente pueda tener infinitos registros a lo largo del tiempo sin comprometer la integridad ni la trazabilidad de su historial clínico.

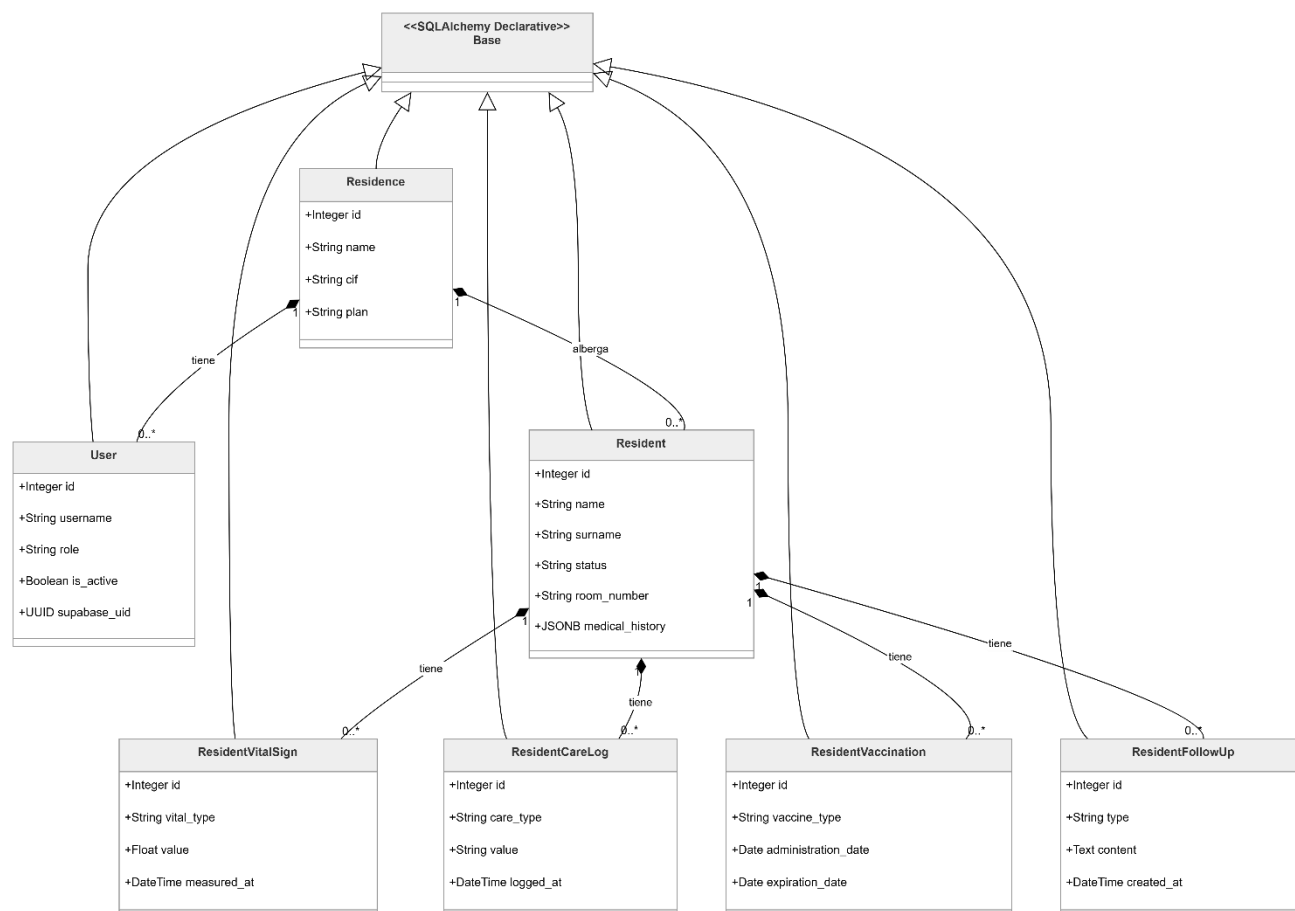
Proyecto “Desarrollo de Aplicaciones Web”

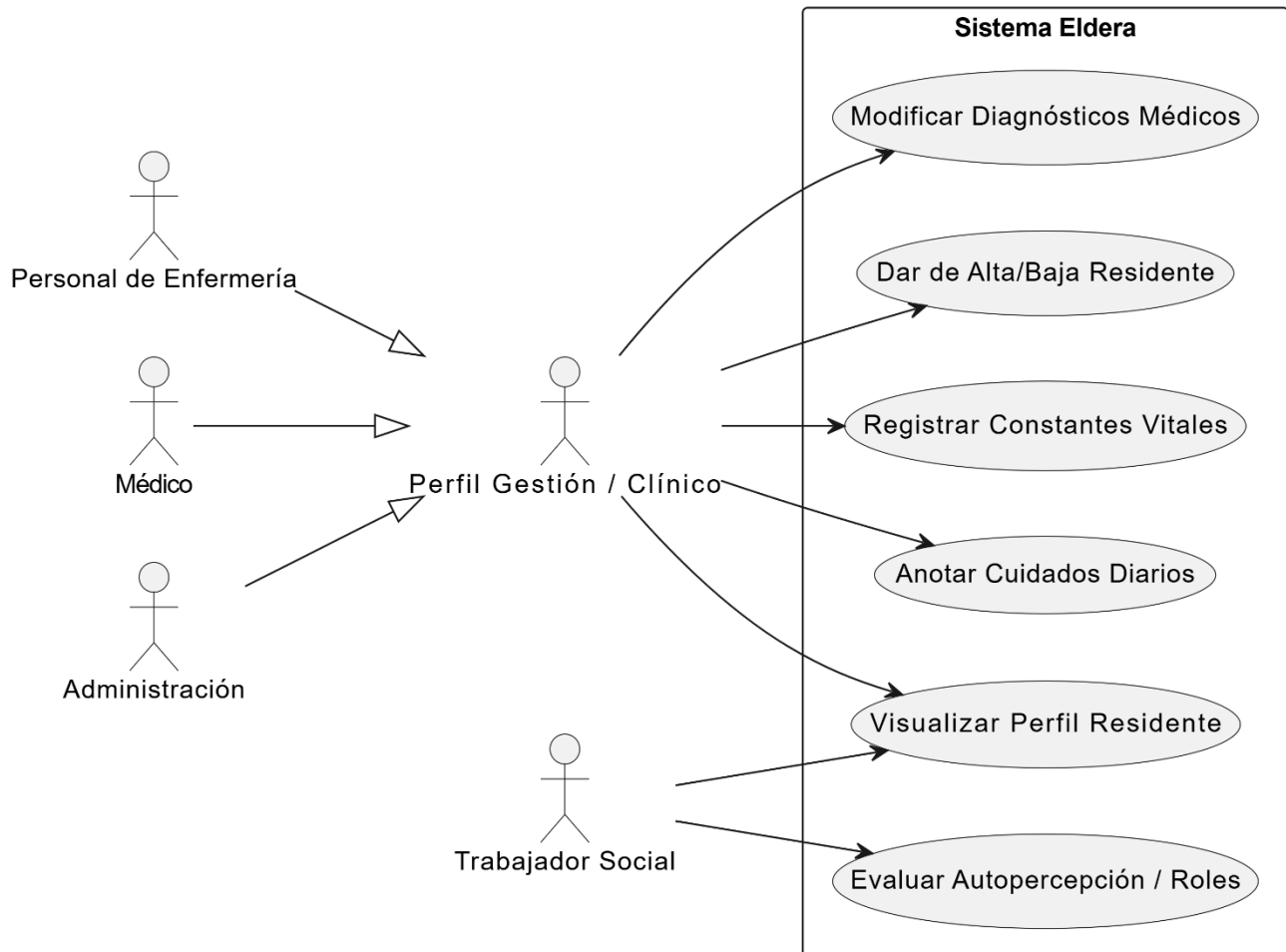


JUNTA DE EXTREMADURA

Consejería de Educación y Empleo

Diagrama entidad-relación simplificando tabla “Resident”.



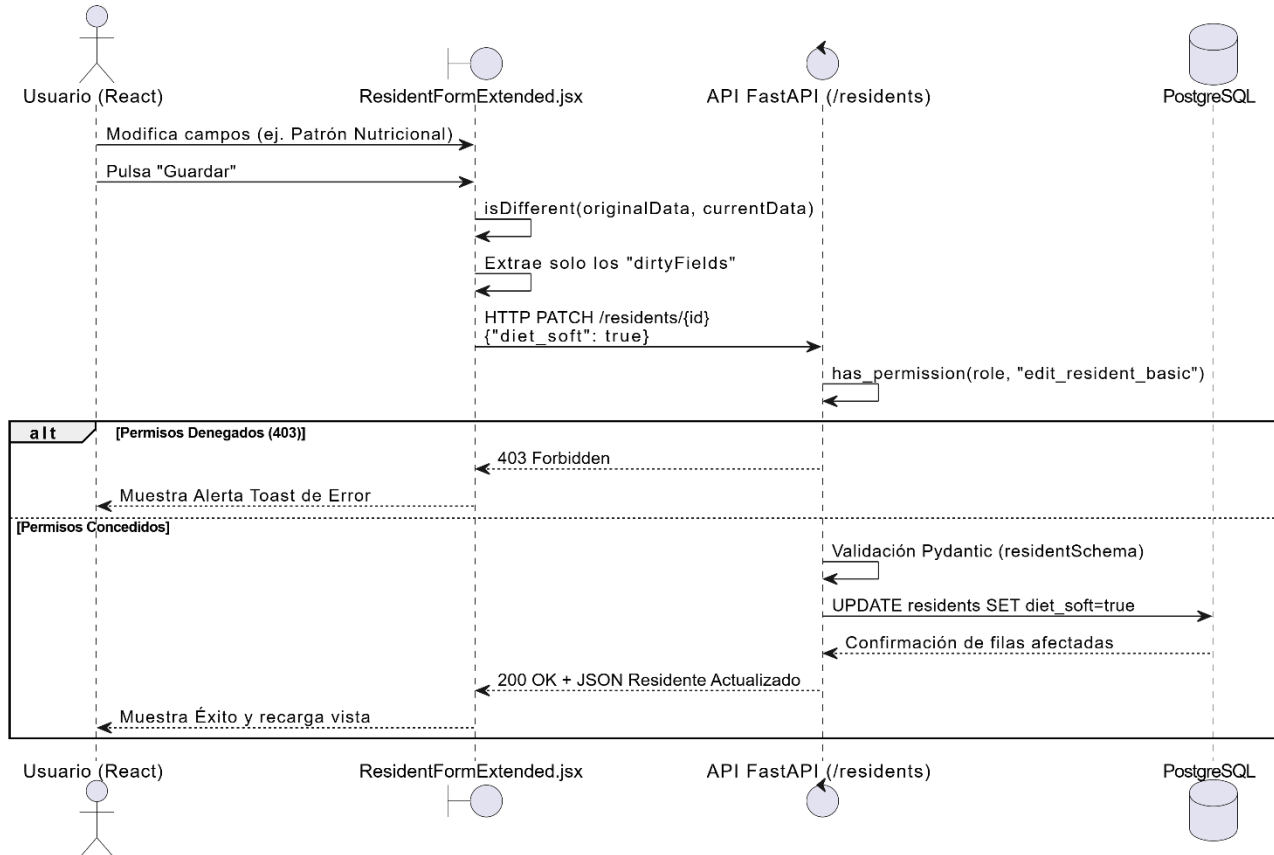


En este Diagrama de Casos de Uso podemos observar cuatro perfiles de nuestro sistema Eldera: Personal de Enfermería, Trabajador Social, Médico y Administración. Como pueden ver, el acceso a la información está democratizado: todos los roles pueden visualizar el perfil del residente, garantizando un flujo de información transparente. Sin embargo, las acciones de escritura están fuertemente tipificadas por el modelo de control de acceso basado en roles (RBAC). El perfil clínico, compuesto por Administradores, Médicos y Enfermería, tiene un amplio nivel de acceso: pueden registrar constantes vitales, anotar cuidados, modificar diagnósticos médicos y gestionar el alta o baja de los residentes. Por su parte, Trabajo Social se enfoca en la valoración de su área (como la autopercepción y el estado cognitivo/social), mientras que Administración tiene un acceso global pero que principalmente se centra en consulta general y gestión de altas/bajas."

Proyecto “Desarrollo de Aplicaciones Web”

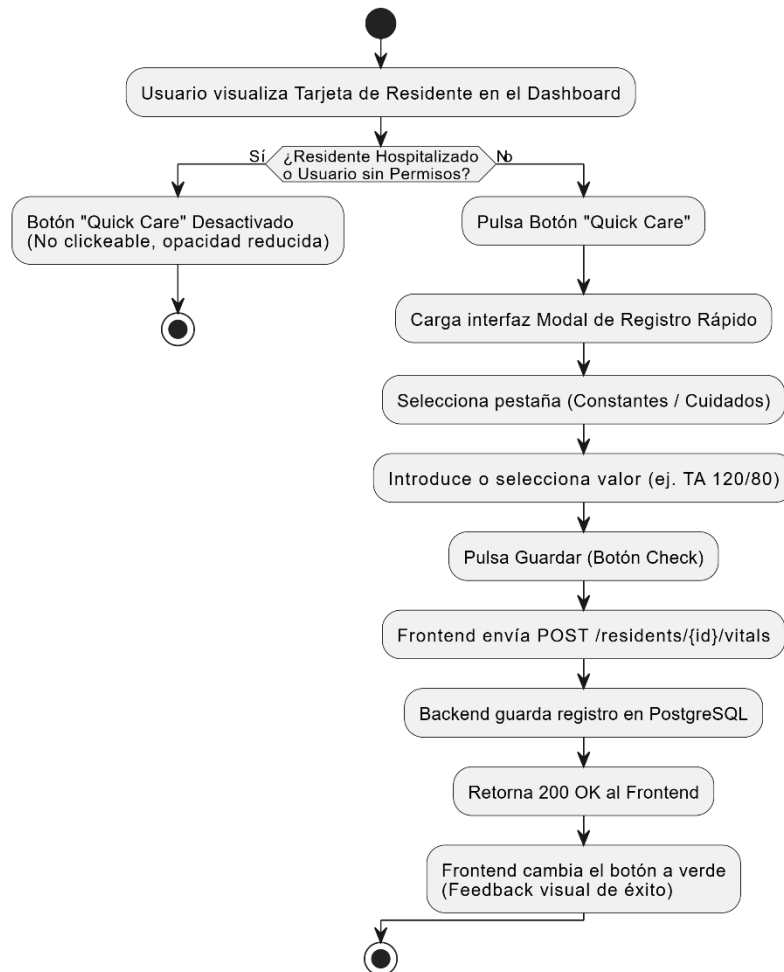


Diagrama A



Si observamos el Diagrama A podemos ver la secuencia del proceso que hemos diseñado bajo el concepto de *Smart PATCH*. Imaginen que un Trabajador Social entra a modificar el patrón de Autopercepción de un residente. Nuestro Frontend en React no envía todo el formulario completo, sino que calcula de forma inteligente qué campos exactos han cambiado (lo que llamamos *dirty fields*).

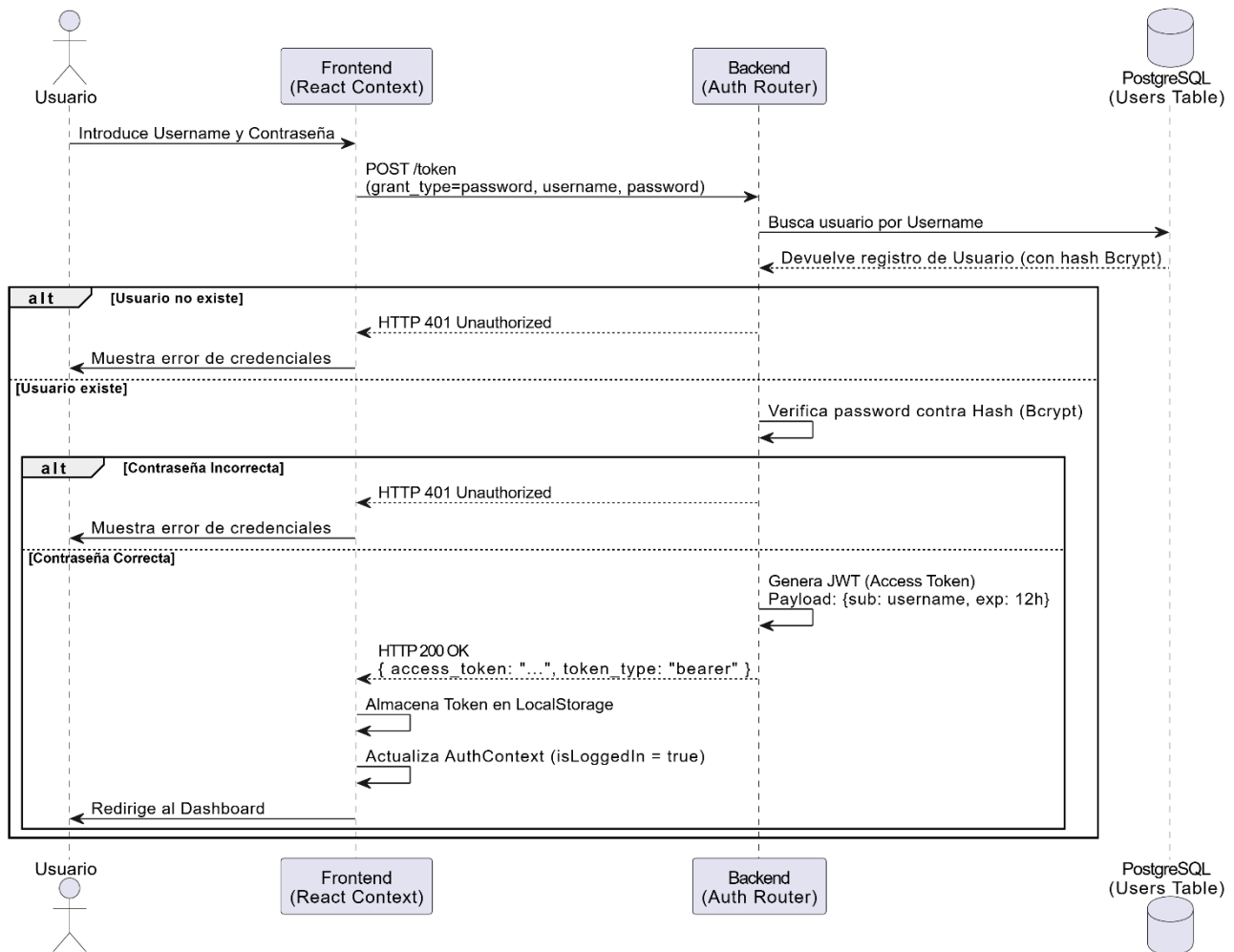
Al pulsar 'Guardar', se envía una petición `PATCH` solo con esos campos. Como vemos en el centro del diagrama, el Backend con FastAPI y Pydantic entra en juego utilizando la propiedad `exclude_unset=True`. Esto es fundamental: le dice al ORM de la base de datos que *únicamente* sobrescriba los datos recibidos. Si en ese preciso instante un médico está guardando nuevas alergias para el mismo residente, sus cambios no se verán machacados, resolviendo así el clásico problema de concurrencia o 'condición de carrera'. Finalmente, el proceso termina devolviendo un 200 OK y mostrando un 'Toast' verde de éxito en pantalla.



Pasando al Diagrama B, aquí ilustramos el flujo de la interfaz Quick Care, nuestro módulo rápido para el día a día.

El flujo comienza en la propia tarjeta del listado de residentes. El sistema hace una doble comprobación inicial en el frontend: si el residente está hospitalizado o si el profesional no tiene permisos asistenciales (como sería el caso de un rol de Trabajo Social), el botón de Quick Care aparece desactivado directamente, impidiendo clics innecesarios y guiando al usuario visualmente. Si el botón está habilitado, el profesional entra a la interfaz modal, elige constantes o cuidados, introduce el dato y guarda. Tras la validación y registro exitoso en la base de datos PostgreSQL, la interfaz devuelve un feedback visual inmediato cambiando el botón a color verde.

Diagrama C



Finalmente, en el Diagrama C, analizamos el corazón de la seguridad de acceso: el proceso de autenticación JWT.

El flujo inicia cuando el usuario introduce su nombre de usuario y contraseña en el Frontend. Estos datos viajan mediante un POST seguro al endpoint `/token` del Backend. FastAPI busca primero si ese nombre de usuario existe en la tabla de Usuarios. Si existe, no compara las contraseñas en texto plano (lo cual sería un riesgo crítico de seguridad), sino que utiliza la librería Bcrypt para verificar si el hash de la contraseña introducida coincide con el hash encriptado de la base de datos.

Si la comprobación es exitosa, el servidor genera un 'Access Token' (JWT) firmado con una clave secreta y un tiempo de expiración (payload). Este token se devuelve a React, donde se almacena en el estado global (Context) y en LocalStorage. A partir de este momento, en la parte final del diagrama, el usuario es redirigido al Dashboard y todas sus peticiones futuras llevarán ese token como su pasaporte de acceso.

3.2. Desarrollo

El desarrollo del API REST sigue convenciones estrictas y asíncronas para no bloquear la ejecución:

- Smart PATCH (Actualizaciones Parciales): Para la edición de perfiles, el backend expone endpoints `PATCH` que, usando `exclude\_unset=True` en Pydantic, permiten actualizar solo los campos enviados desde React. Esto previene la sobreescritura accidental de datos.
- Seguridad y Criptografía:
 - Hasheo Bcrypt: Las contraseñas nunca se almacenan en texto plano. Se utiliza el algoritmo Salted Bcrypt para garantizar que, incluso ante una fuga de datos, las credenciales sean indescifrables.
 - Tokens JWT (Stateless): La sesión se gestiona mediante JSON Web Tokens. Esto permite un backend escalable que no necesita almacenar sesiones en memoria, validando la identidad del usuario en cada petición mediante la firma del token.
- Capa de Permisos (RBAC): La aplicación cuenta con una función inyectable como dependencia en FastAPI (`has\_permission`). El control de seguridad evalúa el rol del usuario (normalizado para evitar fallos tipográficos) frente a una matriz de permisos.
- Componentes UI Dinámicos: En Frontend, componentes como `ResidentFormExtended.jsx` mapean internamente mediante diccionarios qué roles pueden editar qué apartados del perfil (por ejemplo: la sección de Autopercepción está habilitada para `social\_worker`, pero deshabilitada para auxiliares).

3.3. Pruebas realizadas

Se ha utilizado Pytest para la validación automática en el Backend y validaciones manuales asíncronas en el Frontend:

Prueba 1: Control de Acceso (RBAC) - 403 Forbidden:

Proyecto “Desarrollo de Aplicaciones Web”



- Componente / Endpoint: `PATCH /api/v1/residents/{id}`.
- Juego de entrada: Petición HTTP desde un rol técnico (ej. `social\_test`) intentando modificar un parámetro restringido (constantes vitales).
- Salida esperada: Código HTTP 403 (Forbidden) con mensaje indicando falta de permisos. El frontend captura el error e impide la visualización blanca.
- Herramienta: Peticiones automáticas asíncronas (Axios / Pytest).

Prueba 2: Guardado Parcial del Perfil Multidisciplinar:

- Componente: `ResidentFormExtended.jsx`.
- Juego de entrada: El usuario modifica un único patrón funcional y pulsa "Guardar".
- Salida esperada: Animación de carga UI; el frontend serializa solo los campos cambiados y envía la petición HTTP PATCH; el servidor responde 200 OK y el sistema de alertas muestra un éxito visual en pantalla.
- Herramienta: React DevTools y Monitor de Red.

4. Proceso de despliegue

El proyecto está dockerizado, lo que abstrae y estandariza el proceso de despliegue en cualquier entorno compatible.

4.1. Software Necesario:

1. Docker Engine (20.10+ recomendado)
2. Docker Compose (v2.0+)

4.2. Despliegue Local (Desarrollo y Testeo):

1. Clonar el código fuente del repositorio.

Web”

2. Iniciar el entorno contenedorizado ejecutando: ``docker-compose up -d --build``. Esto pondrá en marcha la Base de datos PostgreSQL, el servicio FastAPI y el entorno Vite de React.
3. Cargar la estructura de la base de datos y los datos semilla (usuarios y residentes de prueba): ``docker exec -it eldera_daw_backend python src/backend/init_db.py``.
4. Acceder a la interfaz web en ``http://localhost:5180``.

4.3. Despliegue en Producción (Cloud - Render):

La aplicación está preparada para su despliegue automatizado como Infraestructura como Código (IoC) a través del archivo ``render.yaml``:

- Backend: Se despliega como Docker Web Service.
- Frontend: Se compila y sirve estáticamente para máxima eficiencia.
- Configuración: Es esencial declarar en el panel del hosting las variables ``DATABASE_URL`` (conexión a la base de datos de producción administrada) y ``SECRET_KEY``.

5. Propuestas de mejoras

Para el futuro ciclo de vida y expansión de Eldera, se proponen las siguientes actualizaciones:

- 1. WebSockets para Tiempo Real:** Implementación de protocolos bidireccionales con FastAPI para que las notificaciones de enfermería y nuevas lecturas de constantes aparezcan en las pantallas y tablets instantáneamente sin recargar.
- 2. Módulo PWA (Progressive Web App):** Habilitar el caché local mediante Service Workers en el frontend, garantizando el registro de datos incluso si el profesional pierde cobertura Wi-Fi temporalmente en un pasillo de la residencia.
- 3. Integración IoT Clínica:** Consumo de datos en segundo plano a través de APIs de fabricantes de tensiómetros y pulsioxímetros bluetooth, ahorrando entrada manual de datos y previniendo errores humanos.
- 4. Generación de Informes PDF:** Desarrollo de un módulo en el backend capaz de exportar perfiles funcionales completos en un documento médico formal (PDF) para acompañar a los residentes en derivaciones hospitalarias de emergencia.

6. Bibliografía

- Documentación oficial de React: https://react.dev
- Documentación oficial de FastAPI (Construcción de APIs modernas en Python): https://fastapi.tiangolo.com
- Documentación oficial de SQLAlchemy (Object Relational Tutorial): https://docs.sqlalchemy.org
- Documentación oficial de Tailwind CSS: https://tailwindcss.com